

DUOC UC

INFORME INDIVIDUAL

Proyecto: Sistema de Reserva de Canchas Deportivas

Alumno: Abraham Puente

Profesor: Carlos Martínez

Asignatura: Desarrollo Full Stack I

Fecha: 13 Julio 2026

Santiago, Chile

1. Introducción

El proyecto consistió en desarrollar el backend de un sistema de reserva de canchas deportivas utilizando una arquitectura basada en microservicios con Spring Boot. Mi participación se centró en dos frentes principales: el desarrollo íntegro de tres módulos específicos del sistema y, fundamentalmente, la resolución de problemas críticos de infraestructura, sincronización y despliegue en la etapa final. Esta instancia de defensa no solo plasma el código escrito, sino también la capacidad de adaptación técnica ante entornos distribuidos complejos y el trabajo bajo estrictas metodologías de control de versiones. En este informe detallaré la creación de mis microservicios asignados y cómo abordé las dificultades técnicas relacionadas con el enrutamiento interno de la red de Docker, la resolución de conflictos de ramas en Git y la conexión exitosa con Eureka Discovery Server.

2. Descripción del proyecto

El objetivo general fue construir un sistema funcional donde cada microservicio administrara una parte específica del proceso de reserva de canchas. El ecosistema creció paulatinamente hasta contar con servicios para administrar usuarios, recintos, canchas, horarios, reservas, pagos, precios, notificaciones, reseñas y mantenimientos. La arquitectura fue diseñada para garantizar alta disponibilidad, escalabilidad y un bajo acoplamiento entre los componentes, permitiendo que cada servicio gestione su lógica de negocio de forma independiente pero perfectamente sincronizada. El software final simula un entorno real de producción a gran escala, logrando cubrir el ciclo de vida completo de una transacción: desde el registro inicial del usuario y la separación del recinto, hasta la alerta automatizada de eventos y la auditoría técnica de los estados de las canchas. Al finalizar el desarrollo, mi misión fue garantizar que las piezas bajo mi cargo se comunicaran correctamente en un entorno contenedorizado, asegurando que toda la arquitectura pudiera levantarse de forma estable.

3. Desarrollo realizado

3.1 Desarrollo de microservicios asignados

Estuve a cargo de la creación e implementación completa de tres microservicios fundamentales para la operativa del negocio: `notificacion-service`, `resena-service` y `mantenimiento-service`.

Para cada uno de ellos, implementé la lógica de negocio, las operaciones de base de datos y la configuración inicial de Spring Boot. El objetivo principal en esta fase fue asegurar que funcionaran de manera independiente y sin errores antes de someterlos a la integración con la red global del proyecto.

3.2 Resolución de problemas de red y configuración en Docker

Durante la fase de integración final de la arquitectura, enfrenté un desafío crítico donde `mantenimiento-service` no lograba registrarse en Eureka. Identifiqué a través del monitoreo de los logs que el servicio estaba intentando conectarse a sí mismo (`localhost:8761`) y siendo rechazado.

Para solucionarlo, reestructuré el archivo `docker-compose.yml`, cambiando la URL por el nombre real del contenedor del servidor (`discovery-service-app`). Además, inyecté la configuración de forma directa mediante la instrucción `command` en Docker para forzar a Spring Boot a reconocer la ruta correcta, puenteando las configuraciones locales.

3.3 Limpieza de caché y reconstrucción de contenedores

Para asegurar que los arreglos de red tomaran efecto, me encargué de depurar el entorno de Docker, ejecutando secuencias de reconstrucción absoluta (`build --no-cache`) para obligar al motor a descartar imágenes antiguas almacenadas en memoria. Esto garantizó que todos los contenedores se levantaran utilizando las versiones más recientes y reparadas del código.

3.4 Resolución de conflictos de sincronización (Git Merge Conflicts)

Me encargué de resolver colisiones críticas en el repositorio al momento de intentar sincronizar el trabajo local con la nube en GitHub. Específicamente, gestioné conflictos en múltiples archivos `Dockerfile`, tomando la decisión de priorizar las versiones entrantes que incluían construcciones *multi-stage* (utilizando imágenes de Maven automáticas) para asegurar compilaciones limpias dentro de los contenedores, mientras protegía mis configuraciones locales de red en los archivos `.yml`.

4. Evidencia de mi participación

A continuación, detallo mis aportes principales divididos por etapa de desarrollo, omitiendo números de commit por motivos de formato:

Etapa	Principales aportes
Desarrollo Inicial	Creación completa desde cero de notificacion-service , resena-service y mantenimiento-service .
Configuración de Infraestructura	Reestructuración de variables de entorno y mapeo de red interno en el docker-compose.yml para enlazar los servicios con Eureka.
Resolución de Conflictos (Git)	Solución de <i>Merge Conflicts</i> en múltiples archivos, unificando versiones locales de red con construcciones <i>multi-stage build</i> de la nube.
Depuración y Testing	Análisis avanzado de logs, limpieza de caché profunda de Docker y validación del registro exitoso de los servicios en Eureka Server.

5. Archivos en los que trabajé

Durante el semestre y la crítica etapa de integración, trabajé directamente en los siguientes componentes:

- [notificacion-service](#) (Código fuente y propiedades).
- [resena-service](#) (Código fuente y propiedades).
- [mantenimiento-service](#) (Código fuente, dependencias y propiedades).
- [docker-compose.yml](#) (Ajustes de red, comandos de arranque forzado y dependencias entre contenedores).
- [Dockerfile](#) (Resolución de versiones para automatización de compilación con Maven).

6. Aprendizajes

Este proyecto fue clave para comprender que el desarrollo de un microservicio no termina en la escritura del código Java, sino que abarca su despliegue, orquestación y monitoreo. Aprendí a profundidad cómo funciona la red aislada de Docker y por qué `localhost` dentro de un contenedor significa algo totalmente distinto que en la máquina local. Esto me enseñó el inmenso valor de saber leer e interpretar correctamente los logs del sistema para diagnosticar fallas de conexión y problemas de infraestructura que a simple vista no son evidentes.

Además, logré perder el miedo a enfrentar conflictos de control de versiones en Git (*Merge Conflicts*), aprendiendo a analizar las marcas de colisión, evaluar qué bloque de código entrante aporta más valor a la arquitectura y fusionar el trabajo colaborativo de forma segura sin romper el ecosistema. También comprendí la importancia de utilizar enfoques como los *multi-stage builds* en los archivos `Dockerfile` para automatizar la compilación con herramientas como Maven, lo que me dio una perspectiva mucho más madura y profesional sobre cómo se preparan y empaquetan las aplicaciones para entornos reales.

7. Conclusión

Participar en este proyecto me otorgó una visión completa y realista de los desafíos que presenta la arquitectura de software distribuida. Desarrollar desde cero `notificacion-service`, `resena-service` y `mantenimiento-service` me brindó bases técnicas sólidas en Spring Boot, diseño de APIs y manejo de bases de datos. Sin embargo, haber librado la batalla final contra los errores de enrutamiento de Docker, la falta de comunicación con Eureka Server y los choques de versiones en Git fue lo que realmente me enseñó cómo se rescatan, depuran y estabilizan sistemas complejos en momentos críticos.

Lograr que el último microservicio se registrara de forma impecable y que el repositorio quedara completamente limpio y sincronizado fue el cierre perfecto para este desafío. Me llevo la gran satisfacción de haber entregado no solo un código funcional, sino una infraestructura interconectada y robusta. Esta experiencia consolida mi aprendizaje del semestre, demostrando que poseo tanto los conocimientos técnicos como la mentalidad analítica y resolutiva necesaria para enfrentar problemas de integración en el mundo laboral profesional.